Seat Scout AI: Project Plan (Vercel + Postgres, solo build)

Oct 10, 2026 · @Zaid Nabi

Build Seat Scout AI in nine phases over about six weeks, on one Vercel project and one free Postgres database, starting with the parser because it is the riskiest part.

1. Goal and learning outcomes

Students look up their exam seat by roll number or by chatting with the seating plan, and an admin uploads, reviews and publishes the morning and evening plans. The whole system runs on free tiers.

Design rules that shape every phase

- Exact lookups (a roll number) use plain SQL, never the LLM.
- The LLM only handles natural-language questions, through tool calling over your own data.
- The PDF is parsed once, at upload, and never at query time.
- Nothing reaches students until the admin has reviewed a validation report and clicked Publish.
- Search keeps working even when the LLM quota is gone.

Why this is a good learning project

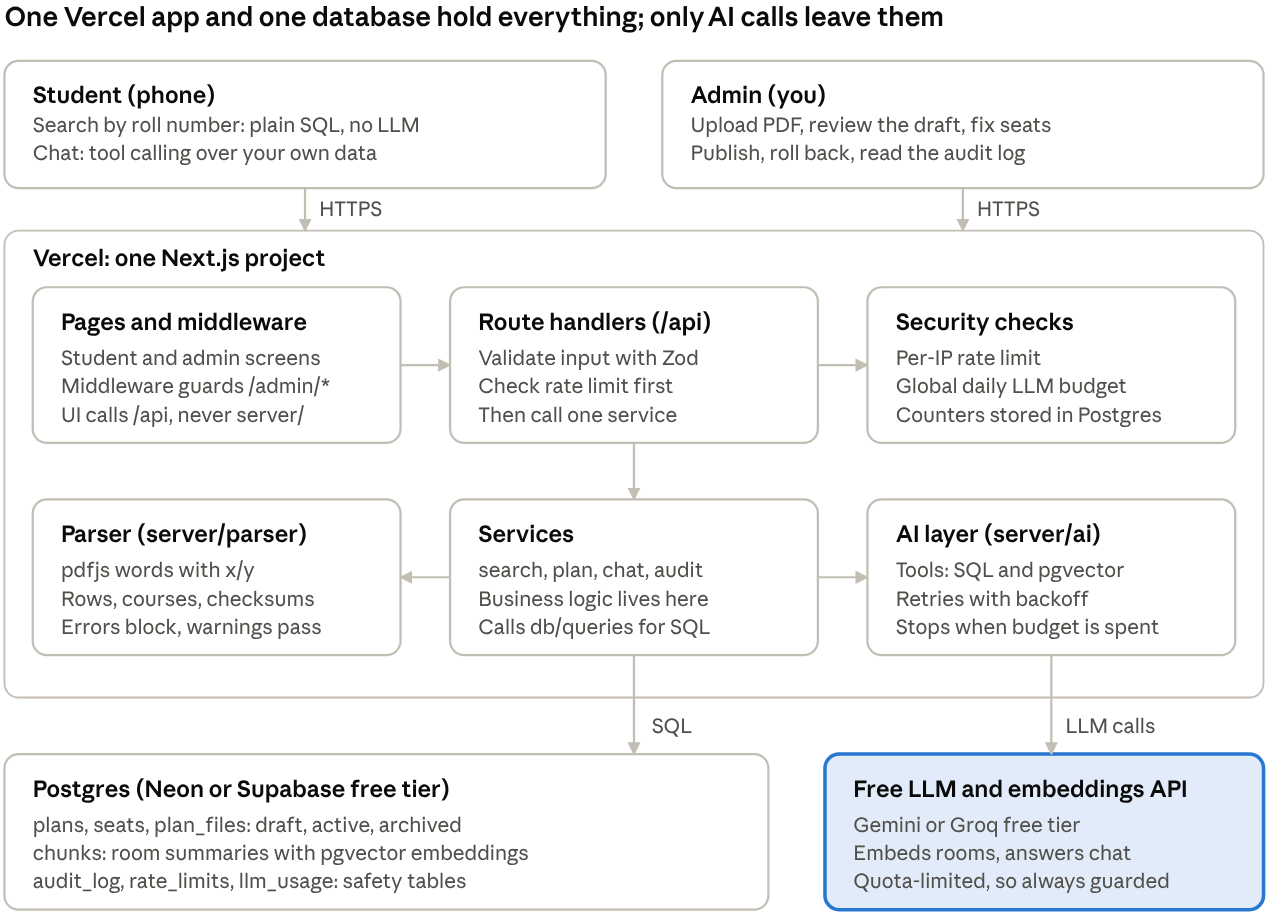
Area	What you practise here
Full-stack	Next.js App Router, route handlers, auth with cookies, forms, file upload, server and client components
Databases	Schema design, indexes, transactions, migrations, pgvector
Data engineering	Layout-aware PDF parsing, validation, checksums
AI engineering	Embeddings, tool calling, grounding, rate and cost limits, graceful fallback

Area	What you practise here
Engineering practice	Testing against fixtures, security basics, free-tier deployment, a README that sells the project

Scope for you as a solo developer. Build the MVP first (phases 0 to 5), then add chat (phase 6), then hardening and deployment (phases 7 and 8). Anything labelled optional can be skipped without breaking the project.

2. Architecture

Students and the admin both talk to one Next.js app on Vercel, which holds all the logic and talks to one Postgres database. The only calls that leave this pair go to a free LLM and embeddings API.



architecture · 1 app, 1 database, 1 external API

Read it top to bottom: a request enters through a page or a route handler, passes the security checks, and reaches a service. The service uses the parser or the AI layer when it needs them, and reads and writes Postgres through the query functions. The AI layer's tools also query Postgres, and only its model and embedding calls go to the external API.

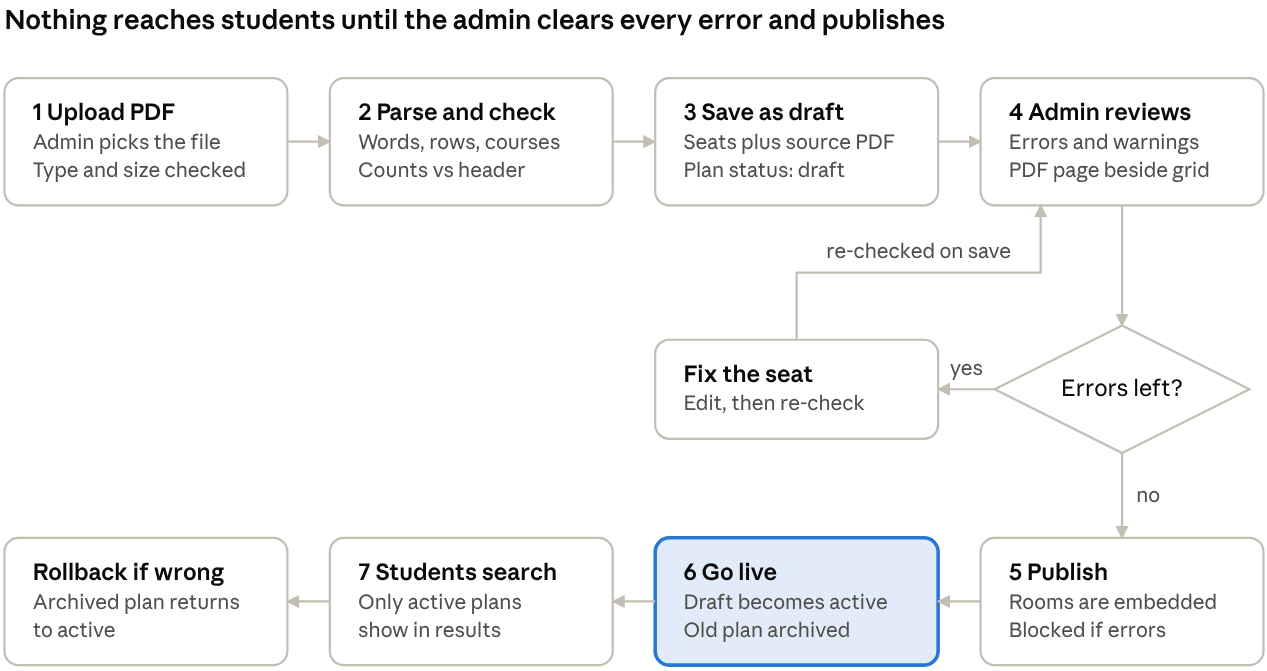
What each part is responsible for

Part	Responsibility	Never does
Pages and middleware	Render screens and block unauthenticated admin requests	Import anything from `server/`
Route handlers	Validate the request, apply limits, call one service, shape the response	Contain business logic or SQL
Security checks	Per-IP limits and the global LLM budget, stored in Postgres because serverless functions share no memory	Trust the client
Services	Business rules: lookups, clash detection, draft and publish, chat loop	Know about HTTP
Parser	Turn PDF bytes into rooms and seats with validation issues	Write to the database
AI layer	Wrap the LLM, embeddings and the four tools	Run model-written SQL
Postgres	Single source of truth for seats, versions, vectors and counters	Run on a schedule you do not control, so a daily cron ping keeps it awake
External API	Embeds room summaries at publish time and answers chat	Receive more than the question needs

3. Full workflow

There are two workflows: the admin turns a PDF into a live plan, and students read that plan through search or chat.

Admin: from PDF to live plan



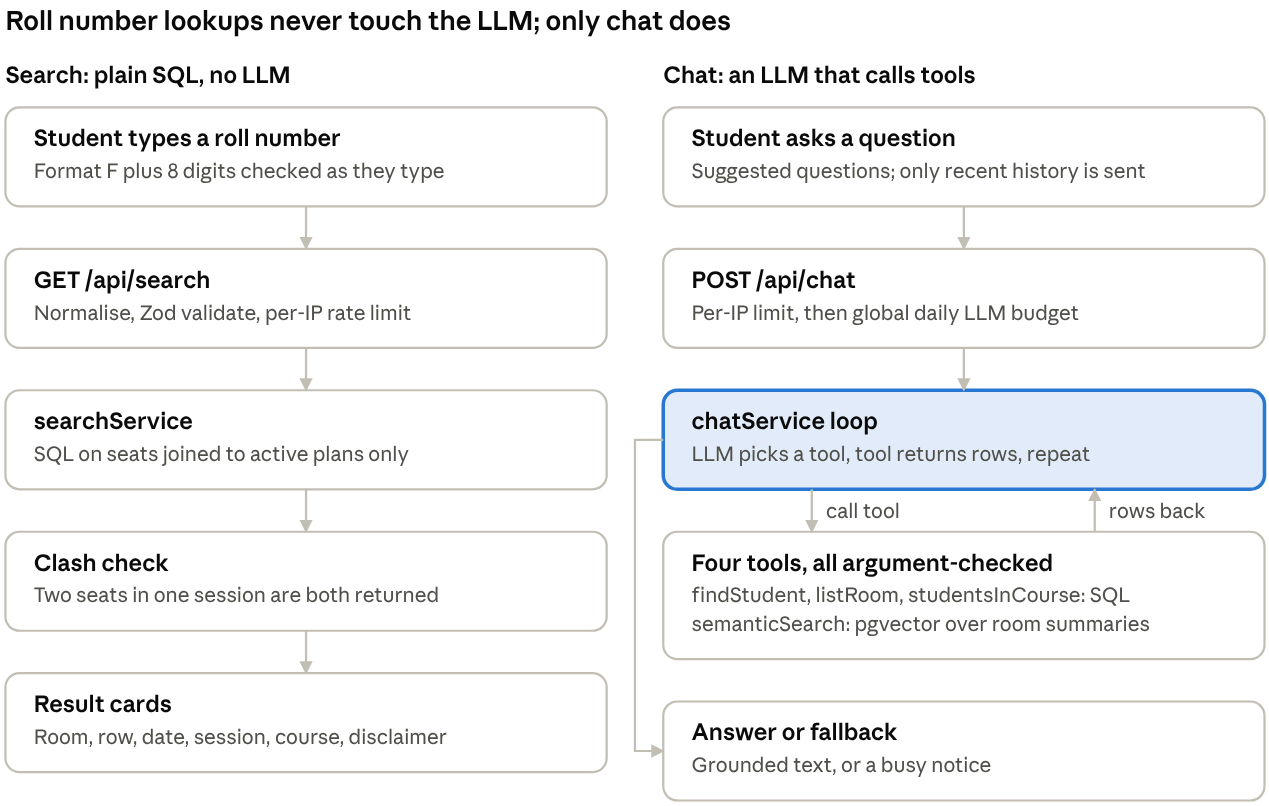
admin workflow, 8 steps, 1 decision, 1 loop

The draft is what makes publishing safe. Students only ever see plans with `status = active`, so a half-finished upload or a failed publish never reaches them.

1. **Upload.** The admin drops the PDF on the upload page. `POST /api/admin/plans` checks that it is a PDF under the size limit.
2. **Parse and check.** `parsePlanPdf` extracts words with x/y positions, splits rooms on the `Room:` header, assigns each seat to Row 1 to 5, and compares counts with the header totals.
3. **Save as draft.** In one transaction the server stores the plan (`draft`), every seat, the page number of each room and the original PDF bytes. The response carries the validation issues.
4. **Review.** The review page shows errors and warnings, the parsed grid for each room, and the original PDF page beside it, so a seat in the wrong row is visible even when the counts match.
5. **Fix.** `ManualSeatEditor` calls the seat PATCH route, which saves the change and re-runs validation. The loop repeats until no errors remain. Warnings, such as a student with two papers in one session, do not block.

6. **Publish.** `activate` refuses if any error remains, embeds the room summaries (the external API call), then in one transaction sets the draft to `active`, the previous plan for that date and session to `archived`, and writes an audit log entry.
7. **Students search.** Search reads only active plans, so the switch is instant and atomic.
8. **Rollback.** If a wrong plan did go live, the admin makes the archived version active again with one click.

Student: search and chat



student workflows, search path and chat path

Search path. Exact and fast: the roll number is normalised, validated and looked up with SQL. A student with two papers in one session gets both seats and a clash warning. An unknown roll number returns an empty list and a helpful message, never an error.

Chat path. The model never sees the whole plan. It decides which tool to call, the tool runs a validated query and returns a few rows, and the model writes an answer from those rows. When the per-IP limit or the daily LLM budget is used up, the API returns a fallback that points to the search box, so the core feature never depends on the LLM.

Why tools and not plain RAG. Embedding chunks of the PDF and retrieving them by similarity is unreliable for exact questions like where F22605010 sits. Tool calling over structured data is exact. Vector search is reserved for fuzzy questions such as which rooms have a given paper.

Background job

A daily cron request to `/api/cron/daily`, protected by `CRON_SECRET`, pings the database so the free tier does not pause, and deletes drafts older than a few days.

4. Tech stack

One Next.js project on Vercel and one Postgres database (Neon or Supabase) cover everything. Free-tier limits change often, so check each provider's pricing page before relying on a number.

Layer	Choice	Why
Framework	Next.js (App Router), TypeScript, Tailwind CSS	UI and API in one deployable project
Database	Neon or Supabase free Postgres	One database for seats, plan versions, rate limits and vectors
Vectors	pgvector extension	No separate vector database to host or pay for
ORM and migrations	Drizzle	Typed queries and plain SQL migrations
PDF parsing	pdfjs-dist (legacy build, no worker) on the server	Gives each word with x/y coordinates, which row assignment needs
Embeddings	Free API (Gemini embeddings or Hugging Face Inference)	A model cannot run inside a free serverless function
Chat LLM	Gemini or Groq free tier with tool calling	Free, rate limited, supports function calls
Admin auth	One admin account, bcrypt hash in env, JWT cookie (jose)	Enough for a single admin, nothing to over-build

Layer	Choice	Why
Validation	Zod	One schema for API payloads and env variables
Tests	Vitest	Fast unit tests for the parser
Hosting	Vercel Hobby plus the free database	No other services

Why the server parses the PDF. Your two plans together are under 200 KB, which is far below Vercel's request-size limit and parses in seconds. That removes the need for browser workers and batch publishing.

5. Folder structure

The code is split by who may import it: `src/lib/` is safe in browser and server, `src/server/` is server-only (every file starts with `import "server-only"`), and components never import from `server/`.

```
seat-scout-ai/
├── .env.example            # DATABASE_URL, GEMINI_API_KEY,
ADMIN_PASSWORD_HASH,
|                           # JWT_SECRET, CRON_SECRET, LLM_DAILY_BUDGET
├── next.config.mjs         # serverExternalPackages: ["pdfjs-dist"]
├── vercel.json             # cron schedule + maxDuration for upload/activate
├── drizzle.config.ts
├── drizzle/                # 0000_init, 0001_pgvector, 0002_plan_files,
0003_audit_ratelimit_usage
├── samples/                # your two PDFs
├── scripts/                # test-parser.ts, seed.ts, reembed.ts
├── docs/                   # architecture.md, api-reference.md, parser-
notes.md, demo.gif
└── src/
    ├── middleware.ts       # guards /admin/* and /api/admin/*
    ├── app/
    |   ├── (student)/      # page.tsx, search/page.tsx, chat/page.tsx
    |   ├── admin/
    |   |   ├── login/page.tsx
    |   |   └── (protected)/    # layout, dashboard, upload, plans/[planId],
audit
    |   └── api/
```

```
    |       ├── search/route.ts
    |       ├── plans/latest/route.ts
    |       ├── chat/route.ts
    |       ├── cron/daily/route.ts
    |       └── admin/          # login, logout, audit, plans/... (upload,
file,
    |                           #   activate, rollback, seats/[seatId])
    ├── components/
    |   ├── ui/             # Button, Input, Card, Badge, Spinner, Modal,
Toast, Skeleton, Disclaimer
    |   ├── layout/         # Navbar, Footer, ThemeToggle, AdminSidebar
    |   ├── search/         # SearchBar, SeatResultCard, SeatResultList,
ClashWarning,
    |   |                   #   LatestPlanInfo, NoResult
    |   ├── chat/           # ChatWindow, ChatMessage, ChatInput,
SuggestedQuestions,
    |   |                   #   TypingIndicator, ChatUnavailable
    |   └── admin/          # LoginForm, StatsCards, PdfUploader,
ValidationReport,
    |                       #   ParsePreviewTable, RoomPreviewCard,
PdfPageViewer,
    |                       #   ManualSeatEditor, PublishButton, PlansTable,
    |                       #   VersionHistory, AuditLogTable
    ├── hooks/              # useSearch, useChat, useUpload, usePlans,
useDebounce
    ├── lib/                # api/ (fetch wrappers), schemas.ts,
normalizeRoll.ts, dates.ts, constants.ts, cn.ts
    ├── server/             # server-only code
    |   ├── parser/         # index, extractWords, splitRooms, parseHeader,
assignRows,
    |   |                   #   parseSeatCell, validate, patterns, types
    |   ├── db/             # client.ts, schema/ (plans, seats, planFiles,
chunks, auditLog,
    |   |                   #   rateLimits, llmUsage), queries/
    |   ├── services/       # searchService, planService, embeddingService,
chatService, auditService
    |   ├── ai/             # llm.ts, embeddings.ts, buildChunks.ts, tools/
(4 tools), prompts/system.ts
    |   ├── auth/           # password.ts, session.ts, requireAdmin.ts
    |   ├── security/       # rateLimit.ts, llmBudget.ts, getClientIp.ts
```

```
    │   └── utils/          # errors.ts, logger.ts, env.ts
    ├── types/              # plan.ts, chat.ts, api.ts
    └── tests/              # fixtures/ (room-301.json, room-302.json),
parser/, services/, api/
```

Layering rule to practise: route handler (HTTP only) calls a service (logic), the service calls a query function (SQL). Each layer can be tested on its own.

6. Database design

Seven tables, all in one Postgres database. Students only ever read rows whose plan has `status = active`.

Table	Key columns	Purpose
`plans`	id, exam_date, session (Morning/Evening), status (draft/active/archived), version, uploaded_at	One row per uploaded plan, with a lifecycle
`seats`	id, plan_id (cascade), roll_no (indexed), course_code, room, row_no (1 to 5), position, program, batch, page_number	One row per student seat
`plan_files`	plan_id, filename, data (bytea)	The original PDF (about 100 KB), so the admin can compare it with the parsed grid
`chunks`	id, plan_id (cascade), room, content, embedding vector(N)	Room and course summaries for semantic search
`audit_log`	id, action, plan_id, at	Who published, rolled back or deleted what
`rate_limits`	key, window_start, count	Per-IP limits that work across serverless instances
`llm_usage`	day, request_count	Global daily LLM budget

Decisions to make before the first migration

1. **Vector size N** depends on the embedding model you pick. Changing it later means re-embedding everything, which is what `scripts/reembed.ts` is for.
2. **Unique (exam_date, session) among active plans**, so only one plan can be live per session.
3. **Indexes:** (plan_id, roll_no) and (plan_id, room) on `seats`. Add an HNSW or IVFFlat index on `chunks.embedding` once real data exists.
4. **Transactions:** activating a plan (archive the old one, activate the new one) must be one transaction so students never see zero or two live plans.

Plan lifecycle

1. Upload creates a `draft` plan with its seats and the original file.
2. The admin reviews the draft and fixes any seat.
3. Publish embeds the rooms, sets the draft to `active` and the previous plan for that date and session to `archived`.
4. Rollback flips an archived plan back to `active`.
5. A daily cron job deletes drafts older than a few days.

7. API reference

Method	Route	Purpose	Access
GET	`/api/search?roll=`	Seat lookup by roll number (active plans only)	Public, rate limited
GET	`/api/plans/latest`	Date and session of the latest active plans	Public
POST	`/api/chat`	Chat with tool calling	Public, rate limited, LLM budget

Method	Route	Purpose	Access
POST	`/api/admin/login`	Admin login, sets JWT cookie	Public, strict rate limit
POST	`/api/admin/logout`	Clear cookie	Admin
GET	`/api/admin/plans`	List plans by status	Admin
POST	`/api/admin/plans`	Upload PDF, parse, validate, save as draft	Admin
GET	`/api/admin/plans/[planId]`	Draft or plan detail with validation issues	Admin
DELETE	`/api/admin/plans/[planId]`	Delete a plan	Admin
GET	`/api/admin/plans/[planId]/file`	Original PDF for the page viewer	Admin
PATCH	`/api/admin/plans/[planId]/seats/[seatId]`	Manual seat correction, re-validates	Admin
POST	`/api/admin/plans/[planId]/activate`	Embed, make active, archive previous	Admin
POST	`/api/admin/plans/[planId]/rollback`	Make an archived plan active again	Admin
GET	`/api/admin/audit`	Audit log	Admin
GET	`/api/cron/daily`	Keep-alive and stale draft cleanup	Cron secret

Rules for every admin route: the middleware blocks unauthenticated requests, and each handler calls `requireAdmin()` again so a middleware mistake cannot expose it. Validate every request body with Zod, and return the same error shape from every route.

8. Phase plan

Each phase has a hat to wear, a target directory, tasks, what you learn, and a check that tells you it is done. Do not start the next phase until the check passes.

Phase 0: Foundation (hat: DevOps)

Technology: Next.js, Git, Vercel, Neon or Supabase **Primary Focus:** A project that builds, deploys and connects to the database **Target Directory:** repository root

Tasks

1. Scaffold the app and install dependencies:

```
npx create-next-app@latest seat-scout-ai --typescript --tailwind --app --src-dir
cd seat-scout-ai
npm i drizzle-orm @neondatabase/serverless pdfjs-dist jose bcryptjs zod server-only @google/generative-ai pino
npm i -D drizzle-kit vitest tsx @types/bcryptjs
```

2. Create the folder tree from section 5 with empty files, so the layout is fixed early.
3. Write `.env.example`, copy it to `.env.local`, and confirm `.env.local` is in `.gitignore` before your first commit.
4. Create the free database, enable the `vector` extension, and link the GitHub repo to a Vercel project.
5. Add `src/server/utils/env.ts`, a Zod schema that fails fast when a variable is missing.

You learn: project setup, environment variables, the client and server boundary, first deployment. **Done when:** a page deploys to Vercel and a test script reads from the database.

Phase 1: Database (hat: database engineer)

Technology: PostgreSQL, pgvector, Drizzle ORM **Primary Focus:** Data model and query layer **Target Directory:** `src/server/db/` and `drizzle/`

Tasks

1. `client.ts`: one pooled connection. Use the pooled connection string, because serverless functions open many short-lived connections.

2. `schema/`: one file per table from section 6 (`plans`, `seats`, `planFiles`, `chunks`, `auditLog`, `rateLimits`, `llmUsage`). Pick the vector size first.
3. Generate migrations with drizzle-kit and run them. Check that `0001` creates the `vector` extension.
4. `queries/seats.ts`: `findSeatsByRoll(roll)` (active plans only), `listRoom(planId, room)`, `studentsInCourse(code, date)`.
5. `queries/plans.ts`: create draft, get, list, delete, `activatePlan` (one transaction), `rollbackPlan`.
6. `scripts/seed.ts`: insert one fake plan so you can test search before the parser exists.

You learn: relational schema design, indexes, transactions, migrations, the difference between a connection and a pool. **Done when:** the seed script runs and `findSeatsByRoll` returns the fake student in under 100 ms.

Phase 2: Parser (hat: data engineer)

Technology: TypeScript, pdfjs-dist, Vitest **Primary Focus:** Turn the PDF into validated structured data **Target Directory:** `src/server/parser/` and `scripts/test-parser.ts`

This is the hardest and most valuable phase. Build and test it on its own, before any UI exists.

Tasks

1. `extractWords.ts`: use the pdfjs legacy build with the worker disabled, and return `{ text, x, y, page }` for every item. Flip y so the top of the page is 0.
2. `patterns.ts`: roll number `/^F\d{8}$/`, course code `/^[A-Z]{2,4}\d{4}$/`, the `Room: | Date: | Session:` header, and the literal `EMPTY`. Keep the roll pattern configurable.
3. `splitRooms.ts`: group words by page and split on the `Room:` header. Keep the page number for every room.
4. `parseHeader.ts`: return room, ISO date, session, per-course counts and the stated total. Read the session from the header, never from the filename.
5. `assignRows.ts`: find the x positions of the `Row 1` to `Row 5` labels, assign each roll number to the nearest column, and order by y for `position`.
6. `parseSeatCell.ts`: read course code, program and batch below each roll number. Handle both cell shapes (program plus year, and batch code like BSCE-23-27) and `EMPTY` seats.

7. `validate.ts`: compare parsed counts per course and in total with the header (Room 302 must give 25 + 24 = 49). Count mismatches and unknown cell shapes are **errors**. Duplicate roll numbers are **warnings**, because real timetable clashes exist.
8. `index.ts`: export `parsePlanPdf(buffer)` returning date, session, rooms (with seats, header counts and issues) and a summary.
9. Save the expected output for Room 301 and Room 302 as fixtures and write Vitest tests for each parser file.

You learn: working with coordinates, regular expressions, defensive parsing, validation design, testing against fixtures. **Done when:** both sample PDFs parse with zero errors and every room matches its header counts.

Remember: the checksum catches missing students, but not a student placed in the wrong row. That is why the admin review in phase 5 shows the original PDF page beside the parsed grid.

Phase 3: Backend APIs (hat: backend developer)

Technology: Next.js route handlers, Zod, jose, bcryptjs **Primary Focus:** Search, upload and publish APIs, plus admin security **Target Directory:** `src/app/api/`, `src/server/services/`, `src/server/auth/`, `src/middleware.ts`

Tasks

1. `lib/normalizeRoll.ts`: trim, uppercase and validate the roll format, with a friendly error for bad input.
2. `searchService.ts` and `GET /api/search`: validate with Zod, call `findSeatsByRoll`, detect clashes (two seats in the same session), return an array sorted by date and session. An unknown roll returns 200 with an empty array.
3. Auth: `password.ts` compares against the env hash, `session.ts` signs and verifies a JWT in an httpOnly, secure, sameSite cookie, and `requireAdmin.ts` is called inside every admin handler.
4. `middleware.ts` protects `/admin/*` (except login) and `/api/admin/*`.
5. `planService.ts` and `POST /api/admin/plans`: check file type and size, run `parsePlanPdf`, save the draft plan, its seats and the original PDF in one transaction, and return the plan id with its validation issues.
6. `PATCH /api/admin/plans/[planId]/seats/[seatId]`: apply a manual correction and re-run validation.

7. `POST .../activate`: refuse if any error remains, embed the rooms (phase 6), then activate in one transaction. Write an audit log entry.
8. `POST .../rollback`, `DELETE`, `GET .../file`, and the plans and audit list routes.
9. Return the same error shape from every route using the classes in `utils/errors.ts`.

You learn: REST design, input validation, cookie auth, middleware, the service layer pattern, transactions from application code. **Done when:** with curl or Postman you can log in, upload the sample PDF, see its validation issues, activate it and then find a student through `/api/search`.

Phase 4: Student search UI (hat: frontend developer)

Technology: React, Next.js, Tailwind **Primary Focus:** Fast, mobile-first seat lookup **Target Directory:** `src/app/(student)/`, `src/components/ui/`, `layout/`, `search/`, `src/hooks/`

Most students will open this on a phone minutes before an exam, so it must be quick and simple.

Tasks

1. Build the UI kit: `Button`, `Input`, `Card`, `Badge`, `Spinner`, `Modal`, `Toast`, `Skeleton`, `Disclaimer`, plus `Navbar`, `Footer` and `ThemeToggle`.
2. `page.tsx` (home): a large roll number input, a link to chat, and `LatestPlanInfo` showing the date and session of the live plan.
3. `SearchBar`: uppercase as the user types, show an inline format error, submit on Enter, and navigate to `/search?roll=...` so results are shareable.
4. `search/page.tsx`: `SeatResultCard` (room in large type, row, date, session, course), `SeatResultList` for several papers, `ClashWarning` when two seats overlap, and `NoResult` with advice.
5. Hooks: `useSearch` for fetching, loading and error state.
6. Add loading skeletons, dark mode, accessible labels and focus states, and the `Disclaimer` telling students to verify with the official plan.

You learn: server and client components, data fetching, forms, responsive design, accessibility. **Done when:** on a phone, a student gets their room and row in two taps, and the page works with the keyboard alone.

Phase 5: Admin workflow UI (hat: full-stack developer)

Technology: React, Next.js **Primary Focus:** Upload, review and publish **Target Directory:** `src/app/admin/`, `src/components/admin/`, `src/hooks/useUpload.ts`, `usePlans.ts`

Tasks

1. `login/page.tsx` with `LoginForm`, then the protected admin shell with `AdminSidebar`.
2. Dashboard with `StatsCards` (plans, students, rooms, latest upload).
3. `upload/page.tsx` with `PdfUploader` (drag and drop, PDF only, size check). On success it redirects to `plans/[planId]`.
4. `plans/[planId]/page.tsx` is the review screen:
   - `ValidationReport`: errors and warnings, each linked to its room.
   - `ParsePreviewTable`: parsed against expected count per room, with a status badge.
   - `RoomPreviewCard`: the five-column grid for one room.
   - `PdfPageViewer`: an iframe on the file route with `#page=N`, beside the grid, so you can see wrong rows.
   - `ManualSeatEditor`: fix a seat, which re-validates on the server.
   - `PublishButton`: disabled while errors remain, with a confirmation that names the date, session and student count, and a warning when it will replace an active plan.
5. `plans/page.tsx` with `PlansTable` and `VersionHistory` (rollback), and `audit/page.tsx` with `AuditLogTable`.

You learn: protected routes, file upload, complex forms and state, designing a review step that prevents bad data going live. **Done when:** you can upload both PDFs, spot and fix a deliberately broken seat, publish, roll back, and a student then finds the right seat. **This is your MVP.**

Phase 6: AI chat (hat: AI engineer)

Technology: Gemini or Groq free tier, pgvector, Zod **Primary Focus:** A chat that answers accurately and does not waste free quota **Target Directory:** `src/server/ai/`, `src/server/services/chatService.ts`, `src/app/(student)/chat/`, `src/components/chat/`, `src/hooks/useChat.ts`

Tasks

1. `llm.ts`: one wrapper around your provider with a timeout, retries with exponential backoff, and a typed error for quota exhaustion.

2. `embeddings.ts`: `embed(texts[])` with batching. `buildChunks.ts`: one text summary per room, for example *Room 301, 24 June 2026, Morning: CS4111 (25 students) and MGT3301 (25 students), total 50*, and optionally one per course. `embeddingService.ts` runs this inside `activate`.
3. Tools in `ai/tools/`, each with a Zod schema:
   - `findStudent(roll_no)`: SQL, exact
   - `listRoom(room, date?, session?)`: SQL, with a result cap
   - `studentsInCourse(course_code, date?)`: SQL aggregate (rooms and counts)
   - `semanticSearch(query)`: pgvector over `chunks`
4. `prompts/system.ts`: answer only from tool results, say so when a tool returns nothing, never invent seats, and do not dump whole rooms on request.
5. `chatService.ts`: the tool-calling loop with a small maximum number of iterations, using only the last few turns of history.
6. `POST /api/chat`: per-IP rate limit, then the global LLM budget, then the service. On quota exhaustion return a friendly fallback that points to the search box.
7. Chat UI: `ChatWindow`, `ChatMessage`, `ChatInput`, `SuggestedQuestions`, `TypingIndicator`, and `ChatUnavailable` for the fallback, with auto-scroll and a notice that the search page is the source of truth.
8. Optional: let the admin upload notices or exam rules as extra text, chunk them, and store them in `chunks` with a `source` field. This is the classic chat-with-a-document use of vectors.

You learn: embeddings, vector search, function calling, grounding, prompt design, cost control, why exact lookups should not go through RAG. **Done when:** asking where a roll number sits returns the right room and row, asking which rooms have a course returns the right list, and the page degrades gracefully when the LLM is unavailable.

Phase 7: Hardening (hat: QA and security)

Technology: Vitest, Postgres **Primary Focus:** Correctness and abuse protection **Target Directory:** `src/server/security/`, `src/tests/`, `src/server/auth/`

Tasks

1. `rateLimit.ts`: a per-IP counter stored in `rate_limits`, with a generous limit on search, a strict one on chat, and a very strict one on login. In-memory counters do not work on serverless, which is why this lives in Postgres.

2. `llmBudget.ts`: a global daily request cap in `llm_usage` and a circuit breaker that switches chat off cleanly.
3. Tests: every parser file, search with a roll that has two papers, activate refused while errors exist, admin routes returning 401 without a cookie, rollback restoring the previous plan.
4. Security checklist: admin checks in middleware **and** in each handler, httpOnly secure cookies, upload type and size limits, no student names stored, chat cannot bulk-dump a plan, no secrets in Git.
5. Test on a real phone with a slow connection, and with the next semester's PDF to see how sensitive the parser is to layout changes.

You learn: testing strategy, rate limiting, threat thinking, defence in depth. **Done when:** the tests pass in CI and you cannot get past any admin route without logging in.

Phase 8: Deploy and present (hat: release)

Technology: Vercel, Neon or Supabase, GitHub **Primary Focus:** Free production deployment and a strong README **Target Directory:** repository root, `vercel.json`, `docs/`, `README.md`

Tasks

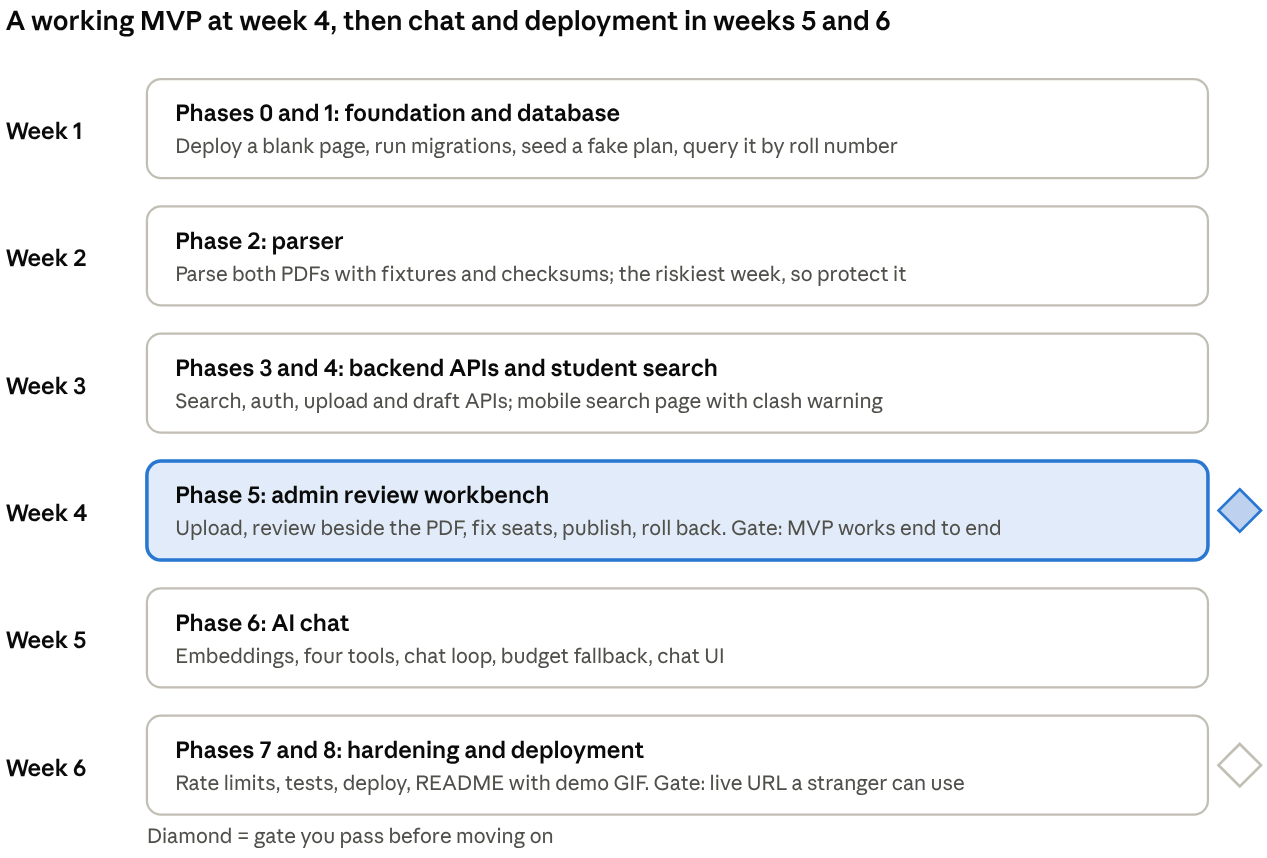
1. Set every environment variable on Vercel, run migrations against the production database, and test the full flow there.
2. Confirm pdfjs works on Vercel: use the legacy build, disable the worker, and keep `pdfjs-dist` in `serverExternalPackages`. This is the most common first-deploy failure, so test it early.
3. `api/cron/daily`: ping the database and delete stale drafts, scheduled in `vercel.json` and protected by `CRON_SECRET`. Check Vercel's current cron limits for the free plan.
4. Set `maxDuration` for the upload and activate routes only up to what the free plan currently allows.
5. README: the problem, a demo GIF, an architecture diagram, setup steps, and the design decisions (why SQL for exact lookups, why tool calling, why draft and publish).
6. Resume bullets, with your own measured numbers:
   - Built a seat-lookup platform with a layout-aware PDF parser that validates every room against header totals.

- Designed a chat assistant using LLM tool calling over SQL and pgvector, keeping exact lookups deterministic.
- Shipped a draft, review, publish and rollback workflow with rate limiting and a global LLM budget, entirely on free tiers.

You learn: deployment, production configuration, writing about technical decisions. **Done when:** a stranger can open the live URL, find a seat, and read in the README why it was built this way.

9. Solo roadmap

Plan for about six weeks of part-time work, with a working MVP at the end of week 4. The weeks are estimates, not measurements: after week 1, replace them with how long things really took.



solo roadmap, 6 weeks, 2 gates

Rules for working alone

1. Finish the parser before you build any screen. A beautiful UI on wrong data is worse than no UI.

2. Work in thin vertical slices. Get one roll number from the database to the screen early, then widen.
3. Commit at the end of every task, and tag the MVP so you can always return to a working version.
4. If a week slips, cut optional items first: theme toggle, audit page, notice upload for chat. Never cut the tests or the review step.
5. Keep a short learning log. After each phase, write what surprised you and what you would do differently. It becomes the content of your README and your interview answers.

10. Risks and mitigations

Risk	Why it matters	Mitigation
Parser puts a student in the wrong row	The checksum only checks counts, so a wrong row passes	Show the original PDF page beside the parsed grid, and offer manual seat correction
Layout changes next semester	Parser assumptions break	Checksum errors block publishing, and fixtures make regressions visible
pdfjs fails on Vercel	Common first-deploy error	Legacy build, no worker, `serverExternalPackages`, deploy early in phase 0
LLM free quota runs out on exam morning	Chat dies for everyone	Per-IP limits, a global daily budget, a fallback message, and search that never needs the LLM
Traffic spike when exams start	Free database connection and cold-start limits	Pooled connection string, cached search responses, a cron ping against idle pauses
Real timetable clash blocks publishing	One student, two papers, same session	Duplicates are warnings, and the UI shows both seats
A bad plan goes live	Students get wrong seats before an exam	Draft, review, publish, plus versions and rollback

Risk	Why it matters	Mitigation
Roll number scraping	Anyone can loop through roll numbers	Per-IP rate limit on search, no names stored, chat refuses bulk dumps
Scope creep	A solo learner can spend weeks on polish	MVP is phases 0 to 5, and everything optional is labelled
A wrong answer harms a student	Real consequences	Disclaimer on every result and a link to the official plan

11. Skills map and definition of done

Track what you can explain, not only what you built. After each phase, write two or three sentences in your own words about the concept in the second column.

Phase	Concept to be able to explain	Skill track
0	Server and client code boundary, environment variables	Full-stack
1	Indexes, transactions, connection pooling	Backend, databases
2	Coordinate-based parsing, validation with checksums, fixture tests	Data engineering
3	Layered architecture, cookie auth, input validation	Backend
4	Server and client components, accessible mobile UI	Frontend
5	Review-before-publish workflow, state in forms	Full-stack
6	Embeddings, vector similarity, tool calling, grounding	AI engineering
7	Rate limiting on serverless, threat modelling, test strategy	Security, QA
8	Production configuration, trade-off write-ups	Delivery

Definition of done for the whole project

- [ ] Both sample PDFs parse with every room matching its header counts.
- [ ] A student finds a seat in under three seconds on a phone.
- [ ] The admin cannot publish a plan that still has errors, and can roll back a bad one.
- [ ] Chat answers roll number and course questions correctly and degrades gracefully when the LLM is unavailable.
- [ ] The project runs on Vercel and one free database with no paid service.
- [ ] The README has a demo GIF, an architecture diagram and a design-decisions section.

Interview questions this project prepares you for: why exact lookups use SQL and not RAG, how you keep a free LLM quota from running out, how you would prove the parser is correct, and what you would change to support a whole university.